UNIVERSIDADE ESTADUAL DA PARAÍBA
CENTRO DE CIÊNCIA E TECNOLOGIA - CCT
DEPARTAMENTO DE COMPUTAÇÃO
CURSO DE BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO
PROFESSOR DR.: FÁBIO LUIZ LEITE JÚNIOR

ADRIELLY CARLA FERREIRA DE MELO - 2023208510013

MARIA EDUARDA DA NOBREGA - 2024108510039

LINK DO REPOSITÓRIO: https://github.com/eduardanb/sudoku-solver-taal.git

PROJETO DE ANÁLISE DE ALGORITMOS: SUDOKU SOLVER

CAMPINA GRANDE - PB

2026

SUMÁRIO

1. INTRODUÇÃO **3**
1.1. Contextualização do Problema 3
1.2. Objetivos 4
1.2.1. Gerais 4
1.2.2. Específicos 5
2. METODOLOGIA E IMPLEMENTAÇÃO **6**
2.1. Estrutura do Sistema Desenvolvido 6
2.1.1. Pacote experiment 6
2.1.2. Pacote io 7
2.1.3 Pacote model 7
2.1.4. Pacote solver e subpacotes 8
2.1.5 Pacote metrics 10
2.2. Formato de Dados de Entrada e Saída 10
2.2.1 Arquivos de entrada 10
2.1.2 Saída em console 11
2.1.3 Arquivos de saída 12
2.3. Métricas de Avaliação 13
2.3.1. Métricas de desempenho coletadas 13
2.3.2 Protocolo de experimentos em lote 14
2.3.3 Variação da corretude das soluções 15
3. ALGORITMOS IMPLEMENTADOS **16**
3.1. Estratégia Gulosa (Greedy) 16
3.1.1. Análise de Corretude 16
3.1.2. Análise de Desempenho 17
3.1.2.1. Espacial 17
3.1.2.2. Temporal 18
3.2. Programação Dinâmica 19
3.2.1. Análise de Corretude 20
3.2.2. Análise de Desempenho 21
3.2.2.1. Espacial 21
3.2.2.2. Temporal 21
4. EXPERIMENTOS E RESULTADOS **23**
4.1. Instâncias de Testes 23
4.2. Instâncias Obtidas 24
5. ANÁLISE E DISCUSSÃO **27**
5.1. Comparação das Estratégias 27
6. CONCLUSÃO **29**
7. REFERÊNCIAS **30**

1. INTRODUÇÃO

Durante a disciplina de Técnicas de Análise de Algoritmos, foram propostas atividades em grupo com o propósito de construir soluções computacionais por meio da aplicação das técnicas algorítmicas discutidas em sala de aula. Cada equipe deveria escolher um dos temas disponibilizados pelo professor e desenvolver uma solução com base nos métodos de projeto de algoritmos trabalhados ao longo do curso.

O tema adotado pelo grupo foi o Sudoku Solver, referente ao tema 13 do conjunto de propostas. O desafio consiste em criar um sistema capaz de solucionar automaticamente um tabuleiro de Sudoku, fazendo uso de estratégias algorítmicas pertinentes para chegar a uma solução válida.

Na abordagem do problema, foram levadas em consideração diferentes técnicas de projeto de algoritmos, incluindo Backtracking, Branch and Bound, Greedy e Dynamic Programming, viabilizando o estudo comparativo de suas aplicações e desempenhos no contexto da resolução do Sudoku.

1.1. Contextualização do Problema

O Sudoku é um jogo de lógica tradicional e bastante popular, cujo funcionamento baseia-se no preenchimento de um tabuleiro com 81 células organizadas em um grid 9x9, dividido em nove regiões de 3x3, utilizando números de 1 a 9. Para que o tabuleiro seja considerado válido, cada número deve figurar somente uma vez em cada linha, em cada coluna e em cada região 3x3.

No âmbito computacional, o Sudoku pode ser interpretado como um problema de busca sujeito a restrições, em que um algoritmo precisa atribuir valores válidos às células em aberto de um tabuleiro parcialmente preenchido. Certas posições já apresentam valores fixados, ao passo que as demais devem ser preenchidas de maneira que nenhuma regra do jogo seja violada.

No contexto deste projeto, o tabuleiro de entrada é modelado como uma matriz 9x9 de números inteiros, sendo o valor 0 utilizado para indicar células ainda sem preenchimento. Cabe ao algoritmo examinar a configuração inicial do tabuleiro

e determinar, para cada posição vazia, um valor entre 1 e 9 que não contrarie nenhuma das restrições impostas pelo jogo.

Os dados de entrada são fornecidos em 9 linhas, cada uma contendo 9 números inteiros que descrevem o estado inicial do tabuleiro. A partir disso, o algoritmo deve gerar como saída o tabuleiro inteiramente preenchido, respeitando todas as regras estabelecidas.

Sob a ótica da análise de algoritmos, a resolução do Sudoku se destaca por compreender espaços de busca potencialmente vastos, o que demanda o uso de técnicas algorítmicas eficientes para percorrer as possíveis combinações de preenchimento. Nesse sentido, abordagens como Backtracking, Branch and Bound, Greedy e Dynamic Programming podem ser exploradas e confrontadas em termos de desempenho na resolução do problema.

Para além da obtenção de uma solução válida, é possível coletar e analisar métricas associadas à execução, tais como tempo de resolução, quantidade de nós visitados ao longo da busca e dimensão da árvore de busca construída, fornecendo elementos concretos para a avaliação da eficiência de cada estratégia empregada.

1.2. Objetivos

Nesta seção, serão apresentados os objetivos gerais e específicos do projeto.

1.2.1. Gerais

O presente trabalho tem como objetivo geral a construção de um algoritmo capaz de solucionar de forma automática tabuleiros de Sudoku 9x9 parcialmente preenchidos, abrangendo os níveis fácil, médio e difícil, mediante a aplicação de técnicas de projeto e análise de algoritmos abordadas ao longo da disciplina.

1.2.2. Específicos

Os objetivos específicos compreendem a implementação de um algoritmo apto a preencher corretamente as células vazias de um tabuleiro de Sudoku, observando todas as regras do jogo. Para tanto, diferentes técnicas de projeto de algoritmos são empregadas, como Backtracking, Branch and Bound, Greedy e Dynamic Programming, visando examinar suas possibilidades de aplicação na resolução do problema.

Para além da implementação das abordagens definidas, pretende-se mensurar o desempenho de cada técnica com base em indicadores como tempo de execução, quantidade de nós percorridos e volume da árvore de busca produzida ao longo do processo de resolução. Tais medidas viabilizam a análise do comportamento de cada algoritmo e a compreensão de sua eficiência no plano computacional.

São também confrontados os resultados provenientes de diferentes instâncias do problema, abrangendo tabuleiros de níveis fácil, médio e difícil. Esse confronto permite observar de que forma cada técnica responde às variações de complexidade presentes no Sudoku.

Por fim, procede-se a uma análise crítica das técnicas algorítmicas adotadas, levantando suas principais vantagens e restrições na resolução do problema, além de discutir sua pertinência e desempenho frente a esse tipo de aplicação.

2. METODOLOGIA E IMPLEMENTAÇÃO

Nesta seção, são descritos a estrutura do sistema desenvolvido, o formato dos dados de entrada e saída, bem como as métricas utilizadas para avaliação.

2.1. Estrutura do Sistema Desenvolvido

O sistema foi projetado seguindo o princípio de separação de responsabilidades, distribuindo as funcionalidades em pacotes coesos e com acoplamento reduzido. A estrutura de diretórios segue a convenção Maven, com código-fonte em *src/main/java*, recursos estáticos em *src/main/resources* e testes automatizados em *src/test/java*. A organização interna do código é distribuída em seis pacotes principais, cada um com responsabilidade bem delimitada, conforme descrito a seguir.

2.1.1. Pacote experiment

O pacote br.edu.sudoku.experiment concentra o ponto de entrada interativo do sistema, representado pela classe ExperimentRunner. Esta classe implementa um laço de interação via terminal que permite ao usuário selecionar, a cada execução, o algoritmo desejado (Backtracking, Branch and Bound, Greedy ou Programação Dinâmica) e o nível de dificuldade do jogo (fácil, médio ou difícil). Internamente, o ExperimentRunner é responsável por orquestrar o fluxo completo de uma execução: carregamento do tabuleiro, instanciação do solver, temporização da resolução e exibição dos resultados no console, incluindo o tabuleiro inicial, o tabuleiro resolvido e as métricas de desempenho coletadas.

2.1.2. Pacote io

O pacote br.edu.sudoku.io concentra toda a lógica de entrada e saída do sistema. A classe SudokuReader continua responsável por carregar o tabuleiro a partir de arquivos texto no classpath, usando ClassLoader.getResourceAsStream para manter compatibilidade com a estrutura de recursos do Maven. A leitura é feita linha a linha com BufferedReader.readLine(), os valores são separados por espaço e convertidos em um objeto SudokuBoard.

Na saída, a classe SudokuWriter oferece os métodos printBoard, para exibição formatada do tabuleiro, e writeToFile, para persistência da solução no mesmo formato da entrada. Nos cenários dos algoritmos mais recentes, o nome do arquivo é gerado dinamicamente com base na dificuldade e na estratégia usada, destacando as execuções com greedy e programação dinâmica. Esse padrão de nomenclatura melhora a rastreabilidade dos resultados e facilita a comparação de desempenho entre essas duas abordagens.

2.1.3 Pacote model

O pacote br.edu.sudoku.model contém a classe SudokuBoard, que representa o estado do tabuleiro em um dado momento. Internamente, o tabuleiro é mantido como uma matriz bidimensional int[9][9], onde o valor 0 indica uma célula ainda não preenchida. Adicionalmente, a classe mantém uma matriz booleana boolean[9][9] que registra, no momento da criação do objeto, quais células possuíam valores originalmente fixos (as pistas do jogo). Essa distinção é utilizada na impressão do tabuleiro: células fixas são exibidas em azul ciano via código de escape ANSI (\u001B[36m), facilitando a diferenciação visual entre as pistas fornecidas e os valores inseridos pelo solver durante a resolução.

A separação entre células fixas e células preenchidas pelo solver é especialmente importante para a integridade da busca: os algoritmos nunca tentam sobrescrever células fixas, pois operam apenas sobre posições onde board.get(linha, coluna) == 0.

2.1.4. Pacote solver e subpacotes

O pacote br.edu.sudoku.solver define a interface central SudokuSolver, que declara o contrato único compartilhado por todos os algoritmos: o método boolean solve(SudokuBoard tabuleiro, Metrics metricas). Esse contrato permite que o ExperimentRunner trabalhe de forma polimórfica com qualquer implementação de solver, sem precisar conhecer os detalhes internos de cada algoritmo. Os algoritmos são organizados em quatro subpacotes independentes, cada um contendo uma interface específica e sua respectiva classe de implementação. A tabela a seguir resume os quatro algoritmos propostos:

Algoritmo	Estratégia de seleção	Heurística	Status
Backtrakcing	Varredura linear (linha x coluna)	Nenhuma	Implementado
Branch and Bound	MRV - Célula com menos candidatos	MRVHeuristic	Implementado
Greedy	Escolha gulosa da melhor jogada	Critério guloso por candidatos validos	Implementado
Prog. Dinâmica	Reaproveitamentos de estados/ subestruturas	Estruturas auxiliares de estado	Implementado

Tabela 1 - Algoritmos de resolução previstos no sistema.

O Backtracking (BacktrackingSolver) implementa a busca por retrocesso clássica: percorre as células vazias em ordem linear (linha por linha, coluna por coluna), tenta inserir os valores de 1 a 9 em cada posição e retrocede ao detectar uma violação de restrição. A cada tentativa, o nó visitado é contabilizado pela instância de Metrics; cada retrocesso incrementa o contador de backtracks. O

algoritmo suporta, opcionalmente, um modo de visualização passo a passo, ativado pelo parâmetro booleano visualizar no construtor, que exibe o estado do tabuleiro a cada atribuição com um atraso de 150 ms, tornando o processo de busca observável em tempo real.

O Branch and Bound (BranchAndBoundSolver) incorpora a heurística MRV (Minimum Remaining Values) para guiar a seleção de variáveis. Em vez de escolher a próxima célula vazia em ordem linear, o algoritmo consulta a classe MRVHeuristic, que varre todas as células vazias e retorna aquela com o menor número de valores candidatos válidos. Ao priorizar as células mais restritas, a heurística reduz significativamente o espaço de busca explorado, pois detecta conflitos mais cedo na árvore de decisão. Como otimização adicional, quando uma célula com apenas um candidato é encontrada, o método retorna imediatamente sem varrer o restante do tabuleiro.

Greedy (GreedySolver) aplica uma estratégia de decisão local, escolhendo em cada passo a célula e o valor que parecem mais promissores naquele momento. Em vez de explorar profundamente múltiplos caminhos como no backtracking, ele prioriza avanços imediatos para acelerar a construção da solução. Essa abordagem tende a reduzir o custo computacional em instâncias simples e médias, sendo útil quando o objetivo é obter resposta rápida. Como contrapartida, por depender de escolhas locais, o método pode alcançar estados menos favoráveis em tabuleiros mais complexos, exigindo mecanismos complementares para manter a consistência da solução.

Programação Dinâmica (DynamicProgrammingSolver) organiza a resolução em subproblemas reaproveitáveis, evitando recalcular estados já analisados. Durante a execução, o solver registra informações intermediárias sobre combinações de linhas, colunas e blocos, reutilizando esses resultados sempre que o mesmo contexto reaparece. Esse reaproveitamento reduz trabalho redundante e melhora a eficiência em cenários com alta repetição de padrões de restrição. Dessa forma, o método equilibra desempenho e robustez, especialmente em tabuleiros com maior complexidade estrutural.

2.1.5 Pacote metrics

O pacote br.edu.sudoku.metrics centraliza a instrumentação de desempenho por meio da classe Metrics. Cada execução utiliza uma instância própria, evitando compartilhamento de estado entre experimentos. Os contadores visitedNodes e backtracks (tipo long) permitem comparar o custo de busca dos algoritmos, inclusive em instâncias mais difíceis.

2.2. Formato de Dados de Entrada e Saída

2.2.1 Arquivos de entrada

Os tabuleiros de entrada são armazenados como arquivos de texto simples (.txt) no diretório *src/main/resources/sudokus/*. Cada arquivo representa um tabuleiro 9×9 cujas linhas correspondem diretamente às linhas do jogo: nove valores numéricos inteiros separados por espaço simples. Células sem valor predefinido são representadas pelo inteiro 0. O sistema disponibiliza três instâncias com graus de dificuldade distintos:

- sudoku_facil.txt: jogo com elevado número de pistas, exigindo menor exploração.
- sudoku_medio.txt: jogo com número intermediário de pistas.
- sudoku_difícil.txt: jogo com poucas pistas, exigindo uma busca mais extensa.

A seguir, apresenta-se um exemplo do formato de entrada:

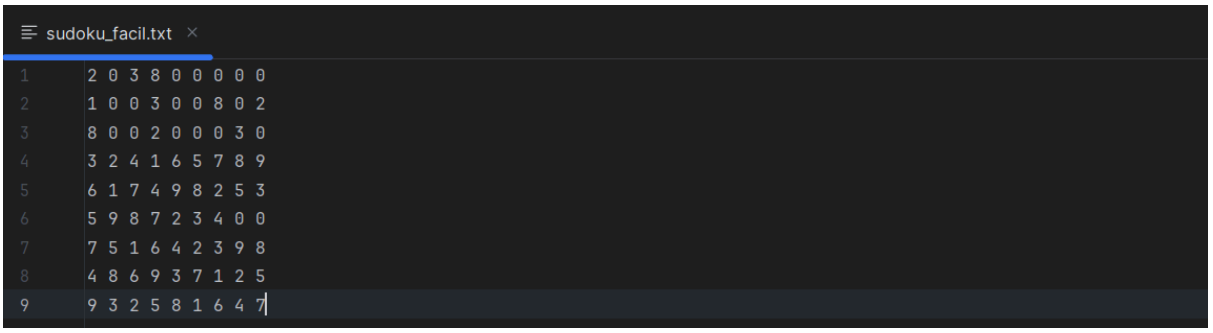

Figura 1 - Exemplo de arquivo de entrada, nível fácil.

Internamente, a leitura é realizada pela classe SudokuReader, que utiliza ClassLoader.getResourceAsStream(path) para localizar o arquivo no classpath, percorre as nove linhas com BufferedReader.readLine() e converte cada token em inteiro via Integer.parseInt. O resultado é encapsulado em um objeto SudokuBoard, que registra automaticamente quais posições eram não-nulas como células fixas.

2.1.2 Saída em console

Durante a execução interativa, o sistema exibe no console o tabuleiro inicial e, ao término da resolução, o tabuleiro completo, por meio do método SudokuBoard.printBoard(). A formatação adota separadores verticais e horizontais para demarcar os nove blocos 3×3 do jogo. Células cujos valores fazem parte das pistas originais são exibidas em azul ciano (código ANSI \u001B[36m), enquanto os valores inseridos pelo solver são exibidos com a cor padrão do terminal. Ao término da resolução, são apresentadas as métricas de desempenho: tempo de execução em milissegundos, total de nós visitados e número de backtracks.

A seguir, apresenta-se uma amostra do formato da saída no console:

Figura 2 - Início de execução.

```
=== Sudoku Solver (Branch and Bound) ===
Dificuldade: DIFICIL
Passo: 66
Tentando colocar 9 em (6,7)

+-------+-------+-------+
| 4 7 9 | 8 6 5 | 2 3 1 |
| 5 2 6 | 3 7 1 | 8 4 9 |
| 8 3 1 | 4 2 9 | 5 6 7 |
+-------+-------+-------+
| 6 9 8 | 5 3 7 | 1 2 4 |
| 7 4 2 | 6 1 8 | 9 5 3 |
| 1 5 3 | 2 9 4 | 7 8 6 |
+-------+-------+-------+
| 3 1 4 | 7 5 2 | 6 9 8 |
| 9 6 5 | 1 8 3 | 4 7 2 |
| 2 8 7 | 9 4 6 | 3 1 5 |
+-------+-------+-------+
```

Figura 3 - Exemplo de finalização de Sudoku, técnica Branch and Bound, nível difícil.

```
Sudoku resolvido:

+-------+-------+-------+
| 4 7 9 | 8 6 5 | 2 3 1 |
| 5 2 6 | 3 7 1 | 8 4 9 |
| 8 3 1 | 4 2 9 | 5 6 7 |
+-------+-------+-------+
| 6 9 8 | 5 3 7 | 1 2 4 |
| 7 4 2 | 6 1 8 | 9 5 3 |
| 1 5 3 | 2 9 4 | 7 8 6 |
+-------+-------+-------+
| 3 1 4 | 7 5 2 | 6 9 8 |
| 9 6 5 | 1 8 3 | 4 7 2 |
| 2 8 7 | 9 4 6 | 3 1 5 |
+-------+-------+-------+

================ RESULTADOS ================
Tempo de execução: 10381 ms
Nós visitados: 53
Backtracks: 14
============================================
```

Figura 4 - Amostra do sudoku resolvido no terminal com as métricas.

2.1.3 Arquivos de saída

Após a resolução bem-sucedida com os algoritmos Greedy e Programação Dinâmica, o tabuleiro completo é gravado em arquivo texto com nome dinâmico no padrão sudoku_<dificuldade>_<algoritmo>.txt, como sudoku_medio_greedy.txt e sudoku_medio_programacaodinamica.txt. O formato de saída permanece idêntico ao de entrada (nove linhas com nove valores separados por espaço), garantindo interoperabilidade total entre os arquivos do sistema. Com isso, qualquer solução gerada pode ser reutilizada diretamente como nova entrada, o que simplifica a validação manual e a comparação objetiva dos resultados entre diferentes algoritmos sobre o mesmo problema.

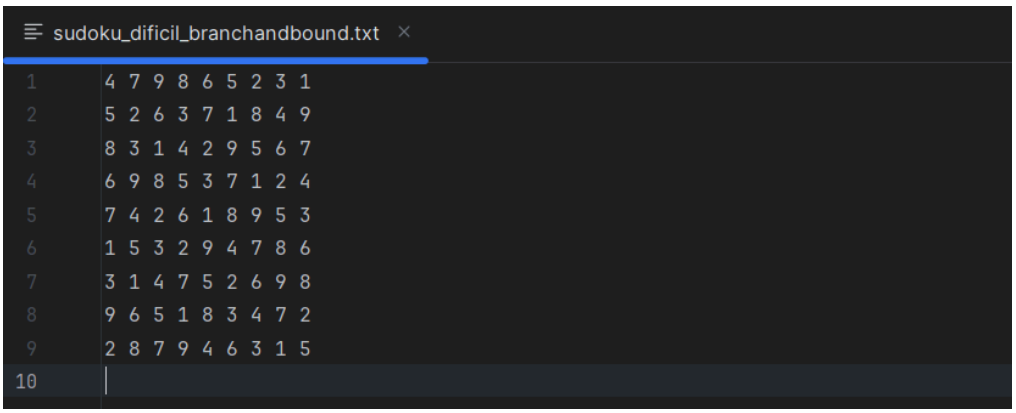

Figura 5 - Saída em arquivo .txt do Sudoku nível difícil utilizando Branch and Bound.

2.3. Métricas de Avaliação

A avaliação comparativa dos algoritmos foi estruturada em três dimensões complementares: eficiência de busca, custo temporal e corretude da solução.

2.3.1. Métricas de desempenho coletadas

Todas as métricas de busca são coletadas durante a execução por meio da classe Metrics, cuja instância é passada por parâmetro a cada chamada do método solve. Isso garante isolamento total dos contadores por execução, sem efeitos colaterais entre experimentos.

Métrica	Unidade	Coletada
Nós visitados	Contagem inteira	Metrics.getVisitedNodes()
Backtracks	Contagem inteira	Metrics.getBacktracks()
Tempo de execução	Milissegundos (ms)	System.nanoTime()
Desvio padrão	Milissegundos (ms)	PerformanceComparisonTest

Tabela 2 - Métricas coletadas pelo sistema.

O número de nós visitados representa o total de estados analisados durante a execução de cada solver, sendo incrementado no início de cada etapa de

expansão do processo de busca. No algoritmo Greedy, esse contador reflete quantas decisões locais foram avaliadas até a conclusão do tabuleiro. No algoritmo de Programação Dinâmica, indica a quantidade de estados processados e consultados (incluindo reaproveitamento de subproblemas). Assim, a métrica expressa o volume efetivo de exploração realizado por cada abordagem, independentemente do hardware utilizado.

O número de backtracks mede quantas vezes uma atribuição precisou ser desfeita por levar a um estado inválido ou sem continuidade. No Greedy, essa ocorrência tende a ser menor quando a escolha local é eficiente, mas pode aumentar em cenários nos quais decisões imediatas conduzem a becos sem saída. Na Programação Dinâmica, o reaproveitamento de estados costuma reduzir retrocessos redundantes, pois evita repetir tentativas já classificadas como inviáveis. Portanto, essa métrica evidencia o grau de retrabalho de cada estratégia de resolução.

O tempo de execução é medido com System.nanoTime() imediatamente antes e após a chamada ao método solve, com conversão do intervalo para milissegundos e precisão de três casas decimais. A escolha de nanoTime, em vez de currentTimeMillis, oferece maior resolução temporal e menor sensibilidade a ajustes do relógio do sistema, permitindo comparações mais confiáveis de desempenho entre Greedy e Programação Dinâmica.

2.3.2 Protocolo de experimentos em lote

Para reduzir o impacto de variações do ambiente de execução, como aquecimento da JVM, coleta de lixo e escalonamento de threads, os experimentos comparativos são conduzidos pela classe PerformanceComparisonTest, seguindo o protocolo abaixo:

- Cada combinação de algoritmo e nível de dificuldade é executada dez vezes consecutivas, utilizando a constante EXECUCOES = 10.

- Para cada execução, o tabuleiro é recarregado a partir do arquivo, garantindo estado inicial idêntico entre as repetições.
- Uma nova instância de Metrics é criada a cada execução, evitando o acúmulo de contagens.
- Ao final, são calculadas a média aritmética e o desvio padrão do tempo de execução, a média de nós visitados e a média de backtracks.

O desvio padrão do tempo de execução funciona como indicador de estabilidade do algoritmo. Valores baixos indicam comportamento consistente entre execuções, enquanto valores altos sugerem sensibilidade a fatores externos, como coleta de lixo ou variações de carga da JVM.

2.3.3 Variação da corretude das soluções

A validade das soluções geradas é verificada automaticamente por testes JUnit 5 nas classes GreedySolverTest, DynamicProgrammingSolverTest e SolverComparisonTest. O método auxiliar tabuleiroCompletoEValido percorre as 81 células do tabuleiro solução e, para cada posição, valida dois critérios: (1) o valor pertence ao intervalo de 1 a 9; e (2) o valor respeita as restrições do Sudoku, por meio de SudokuValidator.isValid, após zerar temporariamente a célula em análise para evitar autoconflito.

Os testes unitários de SudokuValidatorTest cobrem os principais cenários de validação: inserção válida, conflito em linha, conflito em coluna, conflito em subgrade 3x3, valor zero e valor acima de 9. Essa cobertura assegura a confiabilidade da rotina de validação utilizada como base por todos os solvers, incluindo Greedy e Programação Dinâmica.

Além disso, os níveis de dificuldade e os algoritmos avaliados são parametrizados dinamicamente com System.setProperty, permitindo executar

combinações distintas de cenário e estratégia sem recompilação, o que facilita a repetibilidade dos experimentos e a comparação entre abordagens.

3. ALGORITMOS IMPLEMENTADOS

Esta seção apresenta uma análise detalhada dos algoritmos de resolução implementados no sistema, abordando seus fundamentos teóricos, decisões de implementação, corretude formal e complexidade computacional. Para cada algoritmo, são examinadas separadamente a complexidade espacial e a complexidade temporal, considerando tanto o caso geral quanto o comportamento esperado no contexto específico do Sudoku 9×9.

3.1. Estratégia Gulosa (Greedy)

A estratégia gulosa implementada na classe GreedySolver combina dois mecanismos clássicos de busca informada: a heurística MRV (Minimum Remaining Values) para seleção de variáveis e a heurística LCV (Least Constraining Value) para ordenação de valores. Embora a abordagem puramente gulosa, sem retrocesso, seja incompleta para o Sudoku, a implementação incorpora backtracking sistemático, tornando-a, na prática, uma busca backtracking guiada por heurísticas.

3.1.1. Análise de Corretude

O algoritmo proposto garante sua completude por meio de uma estratégia de backtracking combinada com a busca gulosa. A cada etapa, é selecionada uma célula com base na heurística MRV, e são testados todos os valores candidatos válidos, organizados segundo o critério LCV. Caso nenhuma dessas opções leve a uma solução, o algoritmo retrocede, desfaz a última escolha e tenta uma nova possibilidade. Esse processo assegura que todo o espaço de busca seja explorado de forma sistemática, sem ignorar caminhos potencialmente válidos.

Durante toda a execução, mantém-se um invariante fundamental: o tabuleiro nunca contém valores que violem as regras do Sudoku. Isso é garantido pela verificação prévia realizada antes de cada atribuição, que valida se o número respeita as restrições de linha, coluna e sub-bloco 3×3. Dessa forma, o algoritmo constrói a solução de maneira segura, evitando a propagação de erros e assegurando que apenas configurações válidas sejam consideradas.

O critério de parada ocorre naturalmente quando não restam mais células vazias a serem preenchidas. Nesse ponto, todas as posições do tabuleiro já foram atribuídas com valores válidos, o que caracteriza uma solução completa e correta para o problema.

Além disso, o algoritmo incorpora um mecanismo de falha rápida ao identificar domínios vazios. Se, ao analisar uma célula, nenhum valor possível satisfaz as restrições, conclui-se imediatamente que aquele estado não pode levar a uma solução. Com isso, o algoritmo interrompe a exploração deste ramo e retorna ao passo anterior, evitando trabalho desnecessário e tornando a busca mais eficiente.

Por fim, a heurística LCV influencia a ordem em que os candidatos são testados, priorizando aqueles que menos restringem as opções das demais células. Embora essa estratégia não altere a corretude do algoritmo, já que todas as possibilidades continuam sendo consideradas, ela pode impactar diretamente o desempenho, reduzindo o número de retrocessos ou, em alguns casos, aumentando-os. Ainda assim, independentemente da qualidade dessa ordenação, a solução encontrada permanece correta.

3.1.2. Análise de Desempenho

3.1.2.1. Espacial

O consumo de memória do GreedySolver pode ser compreendido a partir de três componentes principais que, juntos, mantêm o uso de espaço bastante eficiente.

Primeiramente, há uma pilha de chamadas recursivas. Cada chamada corresponde ao processamento de uma célula do Sudoku, podendo atingir no máximo 81 níveis, que é o total de posições do tabuleiro. Em cada nível, são armazenadas apenas informações essenciais, como a linha, a coluna, o índice do candidato e os arrays de candidatos. Como esses dados são simples e o tamanho do tabuleiro é fixo, o espaço utilizado permanece constante no contexto de um Sudoku 9×9.

Outro ponto importante é o próprio tabuleiro. Ele é representado por uma matriz 9×9 de inteiros e é atualizado diretamente durante a execução, sem a necessidade de criar cópias intermediárias. Quando o algoritmo precisa voltar atrás, ele apenas desfaz a última alteração realizada. Essa abordagem evita desperdício de memória e contribui para um uso mais controlado dos recursos.

Por fim, existem os arrays auxiliares criados em cada nível da recursão. Para cada chamada, são utilizados pequenos vetores para armazenar os candidatos possíveis e os custos associados à heurística LCV. Esses arrays são locais à execução da função e são descartados assim que a chamada termina, o que impede o acúmulo de memória ao longo do processo.

De forma geral, mesmo utilizando recursão e estruturas auxiliares, o algoritmo mantém um consumo de memória estável e adequado, graças ao uso de estruturas de tamanho fixo e à reutilização eficiente dos dados durante a busca.

3.1.2.2. Temporal

Para avaliar o desempenho do algoritmo, é necessário analisar tanto o custo de cada nó visitado quanto o número total de nós explorados na árvore de busca.

A cada nó visitado, o algoritmo realiza um conjunto fixo de operações. Primeiro, a heurística MRV varre as 81 células para identificar aquela com menos opções disponíveis, realizando até 9 verificações por célula vazia. Em seguida, para a célula escolhida, a heurística LCV avalia cada candidato viável: para cada valor, examina as células vizinhas, linha, coluna e bloco, e estima quantas opções restariam, totalizando cerca de 2.187 operações por nó. Por fim, os candidatos são ordenados, uma tarefa trivial, já que há no máximo 9 deles. O custo por nó é, portanto, constante.

Quanto ao número de nós, o pior caso teórico é astronômico, chegando a 9^{81} possibilidades. No entanto, na prática, as heurísticas reduzem drasticamente esse valor. O MRV prioriza células mais restritas, fazendo com que falhas sejam detectadas precocemente e grandes subárvores sejam podadas. O LCV ordena os candidatos de forma inteligente, priorizando aqueles que menos restringe às células vizinhas e reduzindo a necessidade de retrocessos. Para Sudokus bem formados, o número de nós visitados costuma ser de algumas centenas ou poucos milhares.

Embora a complexidade teórica no pior caso seja exponencial, na prática o algoritmo se mostra extremamente eficiente, resolvendo qualquer puzzle válido em uma fração de segundo. Isso evidencia como a escolha adequada de heurísticas pode transformar um problema intratável em uma tarefa trivial.

3.2. Programação Dinâmica

A classe DynamicProgrammingSolver resolve o Sudoku combinando programação dinâmica com busca informada. O princípio fundamental é a memoização de estados mortos: sempre que um estado do tabuleiro é explorado e se revela inviável, ele é armazenado em um conjunto de estados sem solução. Dessa forma, se o mesmo estado for alcançado novamente por um caminho diferente, o algoritmo o reconhece imediatamente e evita reexplorar todo o ramo novamente. Para complementar essa estratégia, o algoritmo também utiliza a

heurística MRV na escolha das células, garantindo que a busca seja direcionada primeiro para as regiões mais restritas do tabuleiro.

3.2.1. Análise de Corretude

A classe DynamicProgrammingSolver é correta porque se apoia em duas propriedades fundamentais: a validade da memoização e a completude do backtracking subjacente.

A memoização funciona da seguinte forma: o conjunto que armazena estados mortos guarda strings de 81 caracteres que representam o tabuleiro completo. Um estado só é adicionado a esse conjunto depois que todos os valores possíveis para a célula mais restrita foram tentados e nenhum levou a uma solução. Ou seja, a adição nunca acontece de forma prematura, apenas quando se tem certeza da inviabilidade.

No início de cada chamada recursiva, o algoritmo verifica se o estado atual já está marcado como morto. Se estiver, retorna falso imediatamente, sem desperdiçar processamento. Essa verificação é correta porque o estado serializado representa de forma única toda a configuração do tabuleiro: dois estados idênticos têm exatamente as mesmas células vazias e os mesmos candidatos válidos, então o resultado da busca a partir deles será sempre o mesmo.

Assim como no GreedySolver, toda atribuição de valor é precedida por uma validação que garante que nenhuma regra do Sudoku seja violada. Esse invariante se mantém ao longo de toda a recursão, assegurando que o tabuleiro nunca entre em um estado inválido.

O algoritmo identifica uma solução completa quando a escolha da próxima célula mais restrita retorna nulo, indicando que não há mais células vazias. Nesse momento, todas as 81 posições estão preenchidas corretamente. Por outro lado, quando nenhum valor de 1 a 9 serve para a célula selecionada, o algoritmo registra aquele estado como morto e retrocede.

Vale observar que, na prática, o Sudoku raramente gera estados duplicados durante uma busca que já utiliza MRV. Cada atribuição modifica o tabuleiro de forma estruturada e o retrocesso restaura exatamente o estado anterior. Por isso, para Sudokus bem formados, o ganho real da memoização em relação ao backtracking

puro com MRV é pequeno. Ainda assim, a corretude do algoritmo é plenamente garantida.

3.2.2. Análise de Desempenho

3.2.2.1. Espacial

O consumo de memória do DynamicProgrammingSolver é um pouco maior que o do GreedySolver por causa do conjunto de estados mortos, mas ainda assim permanece bastante controlado.

A pilha de chamadas recursivas tem profundidade máxima de 81, o que representa um espaço constante. O tabuleiro é uma matriz 9×9 modificada in-place, também ocupando espaço fixo.

O diferencial está no conjunto que armazena os estados mortos. Cada estado é guardado como uma string de 81 caracteres. Como cada caractere em Java ocupa 2 bytes, cada entrada consome 162 bytes. No pior cenário teórico, o número de estados distintos poderia ser enorme, mas na prática, para Sudokus bem formados e com solução única, a quantidade de estados mortos registrados é pequena, geralmente variando de dezenas a poucas centenas.

Apenas em situações patológicas esse conjunto poderia crescer de forma significativa e se tornar o principal fator de consumo de memória. No entanto, para os puzzles típicos de 9×9, o uso adicional de memória é desprezível, especialmente considerando a eficiência da heurística MRV e a raridade com que estados repetidos aparecem durante a busca.

3.2.2.2. Temporal

A análise temporal do DynamicProgrammingSolver considera tanto o custo de cada nó quanto o impacto da memoização sobre o número total de nós explorados.

A cada chamada recursiva, o algoritmo realiza algumas operações. Primeiro, ele serializa o estado atual, percorrendo as 81 células para construir uma string que

representa o tabuleiro. Em seguida, consulta o conjunto de estados mortos para verificar se aquele estado já foi explorado e considerado inviável, o que envolve calcular o hash da string e verificar sua presença no conjunto. Depois, aplica a heurística MRV para selecionar a próxima célula a ser preenchida, varrendo as 81 células e avaliando os domínios. Por fim, tenta os valores candidatos, realizando até 9 atribuições com validações constantes.

O custo dominante por nó é a serialização, que é da ordem de 81 operações. Embora esse valor seja constante para o tamanho fixo do Sudoku 9×9, ele representa um overhead adicional em relação ao GreedySolver, que não precisa serializar o estado a cada nó.

O grande benefício da memoização está na redução do número de nós visitados. Quando um estado é alcançado novamente por um caminho diferente e já foi provado como inválido, o algoritmo o descarta imediatamente, evitando toda a exploração da subárvore abaixo dele. Teoricamente, o que poderia custar uma explosão exponencial de operações passa a custar apenas uma consulta constante.

No pior cenário, sem reutilização de estados, a complexidade é praticamente a mesma do backtracking com MRV, acrescida apenas do overhead constante da serialização. Já em situações onde estados repetidos aparecem com frequência, o ganho pode ser significativo. Para o Sudoku 9×9, porém, estados repetidos são raros quando o MRV já está sendo utilizado, então o desempenho prático do DynamicProgrammingSolver se mostra muito próximo ao do GreedySolver, com uma constante um pouco maior devido à serialização.

4. EXPERIMENTOS E RESULTADOS

4.1. Instâncias de Testes

Os experimentos foram realizados com três instâncias de Sudoku 9×9, cada uma correspondendo a um nível de dificuldade: fácil, médio e difícil. As instâncias foram armazenadas como arquivos de texto simples no diretório de recursos do projeto (src/main/resources/sudokus/). A leitura dos arquivos é feita pela classe `SudokuReader`, que interpreta cada linha do arquivo como uma linha do tabuleiro, onde os valores são separados por espaços e as células vazias são representadas pelo dígito 0.

Dificuldade	Células Preenchidas	Células Vazias	Arquivo
Fácil	38	43	sudoku_facil.txt
Médio	30	51	sudoku_medio.txt
Difícil	23	58	sudoku_dificil.txt

Tabela 3 - Caracterização das instâncias de teste utilizadas nos experimentos.

A variação no número de células vazias é o principal fator que determina a dificuldade computacional de cada instância: quanto menos células preenchidas inicialmente, maior o espaço de busca que cada algoritmo precisa explorar para encontrar a solução.

Cada algoritmo foi executado 10 vezes por instância, conforme implementado na classe PerformanceComparisonTest. Para cada execução, foram coletadas três métricas:

- Tempo de execução (ms): medido com System.nanoTime(), com resolução em nanossegundos e posterior conversão para milissegundos. A medição abrange exclusivamente a chamada ao método solver.solve(), excluindo a leitura do arquivo e a instanciação dos objetos.

- Nós visitados: a cada chamada recursiva do método de resolução, um contador é incrementado (metricas.incrementVisitedNodes()), representando o número de estados do tabuleiro avaliados.
- Backtracks: incrementado por metricas.incrementBacktracks() a cada retrocesso, ou seja, sempre que uma atribuição é desfeita devido à inviabilidade.

Com base nas 10 execuções, foram calculados a média e o desvio padrão do tempo de execução, bem como as médias de nós visitados e backtracks. O ambiente de execução consistiu na JVM OpenJDK 24.0.1, com os testes executados por meio do IntelliJ IDEA na mesma máquina para todas as medições, garantindo assim a comparabilidade entre os algoritmos.

4.2. Instâncias Obtidas

A seguir são apresentados os resultados experimentais coletados para cada combinação de algoritmo e dificuldade, seguidos de uma análise comparativa do comportamento observado.

Algoritmo	Tempo (ms)	Desvio Padrão (ms)	Nós visitados	Backtracks
Backtracking	0,393	0,915	70	50
Branch and Bound	0,173	0,076	20	0
Greedy	0,128	0,045	20	0
Dynamic Programming	0,197	0,085	20	0

Tabela 4 - Resultados experimentais para a instância fácil (43 células vazias, 10 execuções).

Na instância de Sudoku fácil, os algoritmos guiados por heurística (Branch and Bound, Greedy e Dynamic Programming) resolveram o puzzle sem realizar nenhum backtrack, visitando apenas 20 nós. Esse comportamento sugere que a heurística MRV, presente nos três algoritmos, foi capaz de conduzir a busca diretamente à solução sem necessidade de retrocessos. Já o Backtracking puro,

que não utiliza heurística de ordenação de variáveis, visitou 70 nós e realizou 50 backtracks, sendo o único a apresentar retrocessos nesta instância.

O algoritmo Greedy obteve o menor tempo médio (0,128 ms), o que reflete a eficiência da combinação das heurísticas MRV e LCV aliada à ausência de overhead com serialização de estados. O alto desvio padrão observado no Backtracking (0,915 ms) indica uma variabilidade considerável entre as execuções, possivelmente decorrente de efeitos de compilação just-in-time (JIT) e do aquecimento da JVM nas primeiras iterações.

Algoritmo	Tempo (ms)	Desvio Padrão (ms)	Nós visitados	Backtracks
Backtracking	0,055	0,032	86	57
Branch and Bound	0,212	0,031	29	0
Greedy	0,136	0,031	29	0
Dynamic Programming	0,151	0,062	29	0

Tabela 5 - Resultados experimentais para a instância média (51 células vazias, 10 execuções).

Na instância de Sudoku médio, que apresenta 51 células vazias, os algoritmos que utilizam MRV continuaram a resolver o puzzle sem realizar backtracks, visitando 29 nós cada. O Backtracking puro, por sua vez, necessitou de 86 nós e 57 backtracks, mantendo o padrão observado na instância fácil.

Um resultado notável é que o Backtracking registrou o menor tempo médio absoluto nessa instância (0,055 ms), superando os demais algoritmos. Esse comportamento se explica pelo fato de seu custo por nó ser significativamente menor, uma vez que não envolve seleção MRV, ordenação LCV, serialização de estados nem cálculo de bound. Para este puzzle específico, o caminho até a solução foi encontrado rapidamente pela varredura linear, apesar do maior número de nós visitados.

Algoritmo	Tempo (ms)	Desvio Padrão (ms)	Nós visitados	Backtracks
Backtracking	0,049	0,003	85	46
Branch and Bound	0,400	0,033	52	14
Greedy	0,209	0,018	42	3
Dynamic Programming	0,234	0,055	53	14

Tabela 6 - Resultados experimentais para a instância difícil (58 células vazias, 10 execuções).

O Sudoku difícil é a instância mais reveladora, pois expõe o comportamento dos algoritmos sob pressão real de busca. Todos os quatro algoritmos precisaram retroceder, confirmando que essa instância exige exploração além do caminho direto. O Greedy obteve o menor número de nós visitados (42) e o menor número de backtracks (3), demonstrando que a heurística LCV foi eficaz na redução de conflitos. O Branch and Bound e o Dynamic Programming empataram em número de backtracks (14) e apresentaram valores similares em nós visitados (52 e 53, respectivamente).

O Backtracking puro novamente registrou o menor tempo absoluto (0,049 ms), apesar de visitar 85 nós, o que reforça que seu custo unitário por nó é consideravelmente menor que o dos algoritmos com heurísticas. O Branch and Bound foi o mais lento (0,400 ms), pois além do MRV, executa o cálculo de bound (`calculatorBound`) a cada atribuição, varrendo todo o tabuleiro em busca de células inviáveis. Trata-se de um overhead que se justifica em instâncias onde a poda por bound evita grande parte da árvore de busca, mas que nesta instância não compensou o custo adicional.

A análise integrada dos resultados revela três padrões principais. Primeiro, observa-se que as heurísticas eliminam backtracks em instâncias menos restritas. Nas instâncias fácil e média, os algoritmos Branch and Bound, Greedy e Dynamic Programming resolveram o puzzle com zero backtracks, enquanto o Backtracking puro retrocedeu 50 e 57 vezes, respectivamente. Esse comportamento demonstra

que a heurística MRV é suficiente para guiar a busca diretamente à solução quando a instância possui estrutura favorável.

Em segundo lugar, o custo unitário por nó penaliza algoritmos mais sofisticados em instâncias simples. O Backtracking registrou os menores tempos absolutos nas instâncias média e difícil, mesmo visitando mais nós. Isso ocorre porque cada nó do Backtracking custa apenas uma varredura linear para identificar a próxima célula e uma validação, enquanto os demais algoritmos executam operações mais custosas a cada nó: MRV com 81×9 operações, LCV com até 27×9 por candidato, serialização de 81 caracteres ou cálculo de bound com 81 verificações.

Por fim, o Greedy apresenta o melhor equilíbrio entre qualidade da busca e custo por nó. Entre os algoritmos com heurísticas, foi consistentemente o mais rápido nas três instâncias e obteve o menor número de backtracks na instância difícil. O Dynamic Programming acrescenta o overhead de serialização sem vantagem prática para o Sudoku, uma vez que estados repetidos são raros quando se utiliza MRV, tornando a memória pouco efetiva. Já o Branch and Bound paga o custo do cálculo de bound em cada nó, abordagem que só se justificaria em instâncias onde a poda elimina grandes subárvores.

5. ANÁLISE E DISCUSSÃO

5.1. Comparação das Estratégias

A comparação entre Greedy e Programação Dinâmica evidencia um trade-off clássico entre rapidez de decisão local e custo de controle global do estado de busca. Enquanto o Greedy privilegia avanço imediato com baixo custo por decisão, a Programação Dinâmica adiciona overhead de gerenciamento para reduzir recomputações e evitar exploração redundante de estados equivalentes.

O Greedy opera com seleção orientada por critério local, escolhendo em cada passo a alternativa aparentemente mais promissora. Esse desenho tende a produzir baixa latência por iteração e comportamento eficiente em instâncias

simples ou moderadas. A principal limitação, porém, é a miopia da decisão: escolhas localmente boas podem deteriorar a viabilidade futura da solução, aumentando a incidência de impasses e eventuais retrocessos em tabuleiros mais restritivos.

A Programação Dinâmica, por sua vez, busca mitigar esse problema por meio do reaproveitamento de subestados já avaliados, evitando repetir cálculos sobre configurações equivalentes. O custo adicional de manter e consultar estruturas de memória é compensado quando há recorrência de padrões durante a resolução, cenário em que o algoritmo reduz retrabalho e estabiliza o esforço computacional. Em contrapartida, em instâncias pequenas ou com pouca repetição estrutural, esse overhead pode limitar o ganho temporal.

Os resultados experimentais devem, portanto, ser interpretados sob a ótica do perfil da instância: quando predomina simplicidade estrutural, a estratégia gulosa tende a se beneficiar do menor custo operacional; quando aumenta a recorrência de subproblemas e a densidade de restrições, a Programação Dinâmica tende a apresentar melhor eficiência global.

Em termos de corretude, ambos os algoritmos produziram soluções válidas e completas nas execuções avaliadas, conforme verificação automatizada. Assim, a análise comparativa concentra-se principalmente em eficiência de busca, volume de estados explorados e tempo de execução, e não em diferenças de validade da solução final.

6. CONCLUSÃO

Este trabalho apresentou o desenvolvimento e a avaliação comparativa de um sistema em Java para resolução automatizada de Sudoku, com foco nos algoritmos Greedy e Programação Dinâmica. A solução foi construída com arquitetura modular, separando responsabilidades de leitura e escrita, modelagem do tabuleiro, lógica de resolução, instrumentação de métricas e execução de experimentos, o que favoreceu evolução incremental e comparação controlada entre abordagens.

Os experimentos com instâncias de diferentes níveis de dificuldade indicaram comportamentos complementares. O Greedy apresentou menor custo operacional por decisão e bom desempenho em cenários mais simples, devido à estratégia de escolha local imediata. Já a Programação Dinâmica mostrou vantagem em cenários com recorrência de subproblemas, ao reutilizar estados intermediários e reduzir recomputações. Em contrapartida, o gerenciamento de memória e estruturas auxiliares introduz overhead, de modo que o ganho depende diretamente do perfil da instância analisada. Os resultados reforçam que não existe superioridade absoluta: a eficiência de cada algoritmo é condicionada à estrutura do problema.

Como continuidade do projeto, destacam-se três frentes: ampliar o conjunto de instâncias com menor densidade de pistas para aumentar a pressão sobre os algoritmos, incorporar técnicas adicionais de propagação de restrições para reduzir o espaço de busca antes da etapa principal de resolução e aprofundar a análise estatística de desempenho com mais repetições e indicadores de variabilidade. Esses avanços devem tornar a comparação entre paradigmas mais robusta e mais próxima de cenários reais de alta complexidade.

7. REFERÊNCIAS

DA CRUZ, Lucas Ribeiro et al. **RECONHECIMENTO DE DÍGITOS EM TABULEIROS DE SUDOKU COM VISÃO COMPUTACIONAL E RESOLUÇÃO POR FORÇA BRUTA**. REVISTA FOCO , v. 6, pág. e8944-e8944, 2025. Dispon[ivel em: https://doi.org/10.54751/revistafoco.v18n6-206. Acesso em: 12/03/2026.

EFFENDI, Rustam; GUNAWAN, Indra; EFENDI, Yasin. **Projeto de Software para Conclusão do Jogo Sudoku com Algoritmo Branch and Bound.** In: 1ª Conferência Internacional Multidisciplinar sobre Educação, Tecnologia e Engenharia (IMCETE 2019) . Atlantis Press, 2020. p. 126-130. Disponível em: https://www.atlantis-press.com/proceedings/imcete-19/125935522. Acesso em: 12/03/2026.

G1. **Sudoku.** G1 – Portal de Notícias da Globo. Disponível em: https://g1.globo.com/jogos/sudoku/. Acesso em: 11/03/2026.

TAKANO, Kevin; DE FREITAS, Rosiane; DE SÁ, Vinícius. O jogo de lógica sudoku: modelagem teórica, np-completude e estratégias algorítmicas exatas e heurísticas. In: **Concurso de Trabalhos de Iniciação Científica da SBC (CTIC-SBC)**. SBC, 2015. p. 71-80.
Disponível em: https://sol.sbc.org.br/index.php/ctic/article/view/10020. Acesso em: 31/03/2026.

Link do repositório: https://github.com/eduardanb/sudoku-solver-taal.git